CSC490 Assignment A2

Team: Project 15: Chris Cao, Clarina Ong, Martin Zou, Yanzhen Chen

1 Aspirational Datasets

Imagine you had a magic wand, what datasets would your team need to be successful in this project? Create an aspirational list of datasets with the data schemas you would like.

1.1 Name: Static Dafny Verification Corpus

Purpose:
A deterministic evaluation suite for testing whether the agentic CLI correctly executes Dafny, parses verification results, classifies failures, and localizes errors. This dataset measures baseline correctness of verification reporting.
It evaluates:

- Pass / fail / timeout classification accuracy
- Error classification (e.g., invariant failure, postcondition failure)
- Error localization (line numbers, module identification)

Schema:

```
/case_XXX/
  program.dfy
  metadata.json
```

metadata.json

```
{
  "expected_status": "pass | fail | timeout | syntax_error",
  "error_type": "postcondition_failure | invariant_failure |
                 decreases_missing | quantifier_failure |
                 dependency_missing | none",
  "error_line": 42,
  "timeout_expected": false,
  "description": "Short description of the intended failure mode"
}
```

Programs should include:

- Fully verified programs
- Postcondition violations
- Loop invariant failures

- Missing decreases clauses
- External module resolution failures
- Programs designed to trigger solver timeouts
- Programs with syntax errors

1.2 Name: Incremental Dafny Evolution Dataset

Purpose:
A sequential dataset for evaluating regression detection, state tracking, and verification consistency across iterative edits. This dataset tests whether the CLI can:

- Detect newly introduced verification failures
- Maintain awareness of previously verified components
- Re-run verification correctly after changes
- Update dependency-graph verification statuses

Schema:

```
/sequence_XXX/
  step_0_prompt.txt
  step_0_expected.json
  step_1_prompt.txt
  step_1_expected.json
  ...

{
  "expected_status": "pass | fail | timeout",
  "new_error_introduced": true,
  "affected_modules": ["Stack.dfy"],
  "regression_expected": true,
  "description": "Explanation of expected verification outcome"
}
```

Each sequence begins with a verified baseline program and includes steps that:

- Add new verified features
- Introduce specification mistakes
- Remove invariants
- Add external dependencies
- Introduce solver-heavy constructs
- Repair prior failures

1.3 Name: Dafny Proof Repair Benchmark

Purpose:
A dataset for evaluating the agent's ability to repair verification failures in response to solver feedback. This measures the effectiveness of the generate–verify–repair loop.
It evaluates:

- Repair success rate (does final program verify?)
- Minimality of modification
- Preservation of specifications
- Correct response to error trace

Schema:

```
{
  "input_program": "... Dafny code ...",
  "initial_error_trace": "... Dafny output ...",
  "expected_repair_type": "add_loop_invariant |
                           strengthen_postcondition |
                           add_decreases_clause |
                           modify_spec | other",
  "should_verify_after_repair": true
}
```

Evaluation is behavioral rather than syntactic:

- Whether the final program verifies
- Whether unrelated code was modified
- Whether specification semantics were preserved

1.4 Name: Verification Performance Stress Suite

Purpose:
A dataset designed to measure runtime behavior, solver stability, and timeout classification under varying program complexity.
It evaluates:

- Verification latency
- Timeout detection accuracy
- Stability under repeated runs
- Behavior under solver-stress conditions

Schema:
Directory structure

```
/stress_case_XXX/
  program.dfy
  metadata.json
```

metadata.json

```
{
  "expected_status": "pass | fail | timeout",
  "complexity_level": "small | medium | large | solver_stress",
  "expected_time_bound_seconds": 10,
  "description": "Description of scaling characteristics"
}
```

Programs should vary in:

- Lines of code
- Quantifier complexity
- Number of modules
- Degree of dependency interaction

2 Reality Check

Now review which datasets are actually available. Create a table of datasets that you could use for your project. Include public datasets, data you can scrape or data you can generate. For each dataset include a link to the source, a description of the data and commentary on why it is relevant to your project.

Relevant Item	**Description**	**Commentary**
Official Dafny test suite repo link	Regression tests maintained alongside the Dafny compiler and verifier. The suite contains a large collection of verified programs, intentionally failing examples, edge cases, solver-stress scenarios, and language-feature coverage tests. Each test is designed to validate specific behaviors of the Dafny verifier (e.g., postcondition checking, invariant enforcement, termination reasoning, parsing, and module resolution).	We can directly use this suite as foundational material for our Static Verification Corpus, Proof Repair Benchmark, and Performance Stress Suite. Passing and failing tests provide ground-truth verification outcomes for baseline accuracy measurement. Tests that intentionally fail can be repurposed into repair tasks by pairing them with solver traces. Larger or solver-heavy examples can be adapted for performance and timeout evaluation. Rather than constructing synthetic examples from scratch, we can leverage this real-world corpus to ensure ecological validity while adding structured metadata and sequence evolution layers on top.

Relevant Item	Description	Commentary
HumanEval repo link	Code-generation benchmark evaluated by running unit tests; structurally similar evaluation loop to 'run verifier as oracle" (but uses tests rather than proofs). Each task consists of a natural language function specification and a reference test suite that determines correctness automatically. The benchmark enables objective, programmatic evaluation without human grading.	We can use HumanEval as a structural baseline for our Incremental Dafny Evolution Dataset by preserving its evaluation pattern: use a natural language prompt to generate code, then run an automated oracle check. Instead of unit tests, we substitute the Dafny verifier as the oracle, using pass/fail/timeout outcomes and error traces as ground truth. This allows us to inherit HumanEval's reproducible scoring methodology (e.g., pass@k-style metrics) while extending it to multi-step, regression-aware verification scenarios.
AI-generated evolutionary prompt benchmark	A benchmark consisting of structured sequences of natural language prompts that iteratively modify a Dafny codebase. Each sequence begins with an initial specification (e.g., generate a verified module) followed by successive prompts that introduce new features, refactor components, intentionally inject specification bugs, add external dependencies, or repair prior failures. After each step, the resulting codebase is evaluated using the Dafny verifier as an automated oracle (pass/fail/timeout + error trace).	This benchmark extends HumanEval-style prompt-to-oracle evaluation into a multi-step, regression-aware setting. Unlike static code-generation tasks, it measures an agent's ability to maintain verification correctness over time, detect regressions, and respond to solver feedback. It also allows evaluation of dependency-graph tracking and state-aware verification behavior. By generating controlled prompt sequences, we can systematically test feature addition, failure introduction, and repair dynamics under formal verification constraints. The reason why we chose to use a multi step benchmark process rather than a single prompt and accompanies unit tests is because we want to see how the product reacts after seeing a failing module and correctly adapts to fix it.

Table 1: Relevant benchmarks and datasets for evaluating verification-aware agentic systems.

3 Data-processing pipelines

3.1 Data Schemas

The system treats each agentic CLI execution as a structured event stream. Data is stored in two layers: a raw data lake (immutable JSONL) and a curated warehouse (Parquet tables).

Raw Layer (S3 Dirty Bucket)

```
s3://dirty-bucket/raw/runs/date=YYYY-MM-DD/run_id.jsonl
s3://dirty-bucket/raw/artifacts/run_id/ (diffs, logs, optional snapshot zip)
```

Each JSONL run file contains:

- `run_id` (string)
- `timestamp` (ISO 8601)
- `repo_id` (string)
- `prompt` (string)
- `model` (string)
- `model_params` (json)
- `steps[]` (agent actions)
- `verification[]` (per-module results)
- `duration_ms` (bigint)
- `status` (`pass | fail | timeout | syntax_error`)

Curated Warehouse Tables (Athena / Glue)

runs

- `run_id` (string)
- `timestamp` (timestamp)
- `repo_id` (string)
- `model` (string)
- `status` (string)
- `duration_ms` (bigint)

run_steps

- `run_id` (string)
- `step_id` (int)

- `action` (string)
- `command` (string)
- `stdout_ref` (string)
- `stderr_ref` (string)

verification_results

- `run_id` (string)
- `module` (string)
- `status` (string)
- `time_ms` (bigint)

errors

- `run_id` (string)
- `file` (string)
- `line` (int)
- `error_kind` (string)
- `raw_message` (string)

graph_snapshots

- `run_id` (string)
- `snapshot_ts` (timestamp)
- `nodes` (json)
- `edges` (json)
- `dirty_count` (int)
- `tainted_count` (int)

3.2 Pipeline Diagrams

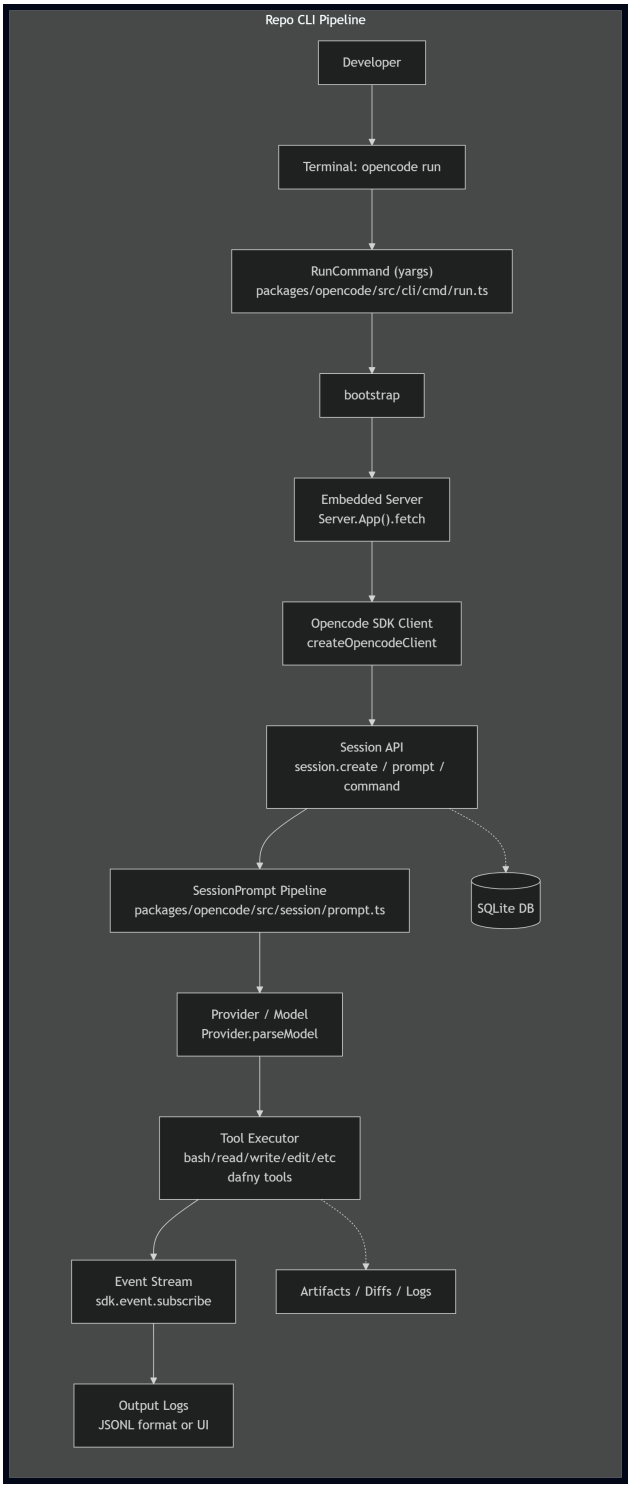

Figure 1: Architecture Diagram

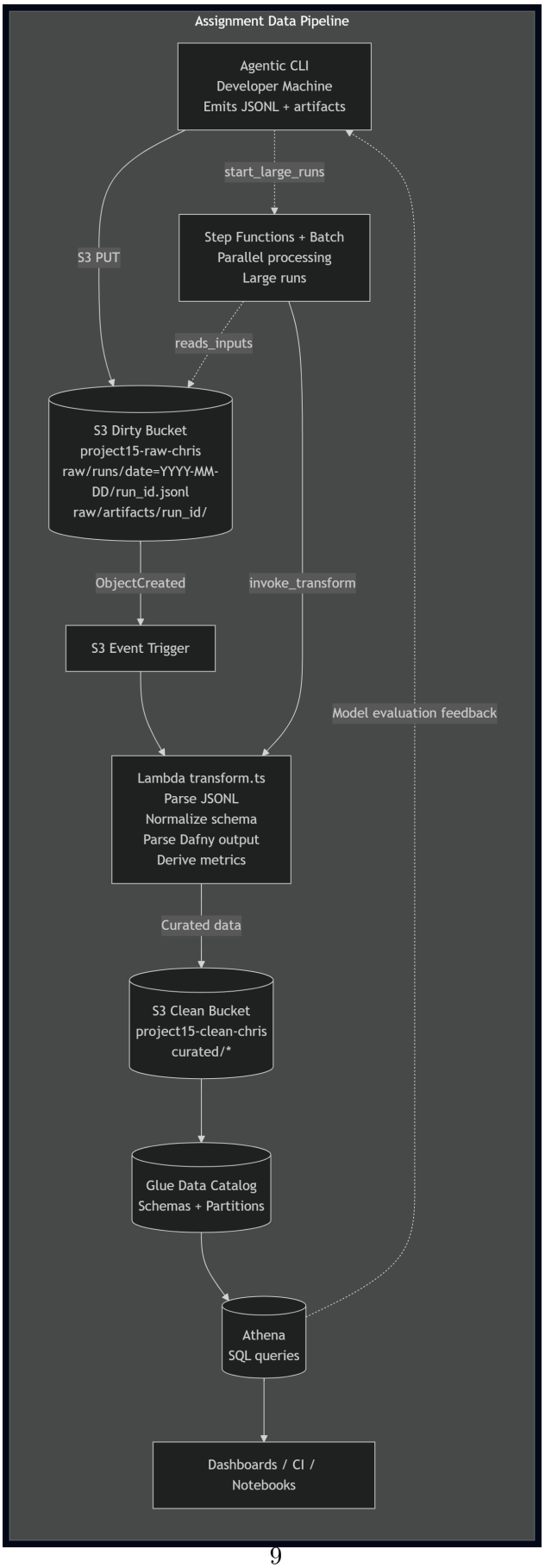

Figure 2: Data Pipeline

3.3 Description of When the Pipelines Will Run and for Which Use Cases

Every CLI run writes structured logs locally and uploads them to the raw S3 bucket for debugging. On new raw uploads, the transformation step parses Dafny output, normalizes schemas, and writes Parquet files to the curated layer. We also use the parsed data during CI to compare pass rates and performance across model configurations.

3.4 Code for an Initial Version of the Pipeline

The minimal initial implementation is in the Github repo under the `pipeline/` folder, but at a high level it consists of the following components:

- `emit_run.ts`: Emits structured JSONL logs and stores artifacts locally.
- `upload_raw.ts`: Uploads run logs to the S3 dirty bucket.
- `transform.ts`: Reads raw JSONL, parses Dafny output, normalizes fields, and writes Parquet files to the clean bucket.
- `schemas.ts`: Zod validation schemas to ensure schema consistency.

3.5 Next Steps for Features Not Implemented in This Writeup

- Automate partition discovery (e.g., Glue crawlers or scheduled `MSCK REPAIR TABLE` in Athena).
- Emit curated outputs as Parquet instead of JSONL for faster analytics.
- Add Lambda/Step Functions for event-driven transforms at scale.
- Implement graph delta storage instead of full snapshots.
- Store diffs rather than full file contents.

4 Infrastructure as Code Implementation

Please check our Github repository under `terraform/` folder.

5 Disaster Recovery Demonstration

The link to a recording of our recovery demonstration is here: link